

CONT_23CSH-314 :: SYSTEM DESIGN

Course Overview	Unit-1 (Theory)	Unit-2 (Theory)
Unit-3 (Theory)	Unit-1 (Practical)	Unit-2 (Practical)
Unit-3 (Practical)		



Announcements



Course Introduction, Objectives and Outcomes

Mark as done

a. Course Introduction

The *System Design* course provides an in-depth study of scalable, reliable, and high-performance distributed software systems. It integrates both **High-Level Design (HLD)** and **Low-Level Design (LLD)** methodologies with core principles of distributed architectures. Students learn key components such as microservices, load balancing, caching, database scaling (replication, sharding), message queues, and fault-tolerant system design. The course emphasizes performance trade-offs to meet non-functional requirements like **latency, throughput, availability, and scalability**, supported by practical case studies such as URL shorteners and chat messaging systems.

b. Course Objectives

- Focus: The course delivers an in-depth study of scalable, reliable, and high-performance system design for modern distributed software systems.
- Methodology: It integrates High-Level Design (HLD) and Low-Level Design (LLD) methodologies.
- Core Topics: Key components covered include load balancers, caching, database scaling (sharding, replication), message queues, CAP theorem, and microservices.

- Outcome: The primary goal is to enable students to design and architect industry-grade systems by analyzing performance trade-offs and meeting Non-Functional Requirements (NFRs) such as latency, throughput, and fault tolerance.

c. Course Outcomes

CO No	Statement	Performance Indicator	Student Outcome Indicator (ABET)	Level of Learning (Highest BT Level)	
CO1	Understand high-level system design concepts including HLD/LLD distinctions, performance requirements (latency, throughput, availability), and scalability challenges.	PI-1a, PI-1b	SO1	BT2 -Easy	€
CO2	Apply object-oriented principles, SOLID guidelines, and UML modeling techniques to design low-level components of software systems.	PI-3b, PI-5a	SO2	BT3 - Medium	€
CO3	Analyze system scalability decisions such as caching, load balancing, microservices communication, and distributed components for large-scale architectures.	PI-2a, PI-4a	SO1,SO2	BT4 - Medium	€
CO4	Evaluate database scalability strategies including sharding, replication, hashing, and event-driven mechanisms based on consistency and performance trade-offs.	PI-2b, PI-6a	SO2,SO6	BT5 -Hard	€
CO5	Design and construct end-to-end distributed architectures for real-world applications ensuring reliability, fault-tolerance, and modern non-functional requirements.	PI-6b, PI-7a	SO2,SO5	BT6 – Hard	€



Course Information

Mark as done

Attribute	Details
Course Title	System Design
Course Code	23CSH-314 / 23ITH-314
Course Type	Hybrid (DE)
Course Category	Graded (GR)
UG/PG Level	Undergraduate (UG)
Number of Credits (C)	3
Lecture Hours (L)	2
Tutorial Hours (T)	0
Practical Hours (P)	2
Total Contact Hours (CH)	4 per week
Pre-Requisites	Basic knowledge of Computer Networks, Databases (SQL/NoSQL), and any one programming language (Java/Python/Node.js)
Co-Requisite	Basic Web Technologies, Understanding of HTTP & REST APIs, Basic Software Engineering Concepts
Anti-Requisite	Nil
Course Instructor / Coordinator	Er. Mayank Sharma



Course Syllabus and Suggestive Readings

[Mark as done](#)

1. Syllabus / List of Experiments

UNIT 1	Fundamentals of System Design	10 Hours	CO Mapping
Chapter 1.1 — Introduction to System Design System Design basics, High-Level Design (HLD) vs Low-Level Design (LLD), Functional vs Non-functional Requirements, Latency, Throughput, QPS, Availability, Scaling challenges. Chapter 1.2 — Object-Oriented Programming Fundamentals (LLD) OOP Fundamentals (Classes and Objects, Enums, Interfaces, Encapsulation, Abstraction, Inheritance, Polymorphism), Class Relationships (Association, Aggregation, Composition, Dependency). Chapter 1.3 — Design Principles and Patterns (LLD) SOLID Principles , Design Patterns Creational (Singleton , Factory Method , Abstract Factory ,Builder) , Structural (Adapter , Decorator , Façade , Proxy), Behavioral (Strategy , Observer) Chapter 1.4 — UML and Software Modeling (LLD)			

Class Diagram, Use Case Diagram, Sequence Diagram, Activity Diagram, State Machine Diagram			
Experiment 1.1	To execute a simple URL Redirection System that converts long URLs into short URLs and measure latency and throughput under different request loads.		CO2
Experiment 1.2	To implement an Object-Oriented Movie Ticket Booking System by using classes and relationships (association, aggregation, composition) to manage movies, shows, seats, and bookings.		CO2
Experiment 1.3	To demonstrate the Low-Level Structure of an Online Movie Ticket Booking System by applying SOLID principles and using suitable Design Patterns such as Factory Method for creating movie objects, Observer Pattern for show seat availability updates, and Strategy Pattern for handling multiple payment methods.		CO1, CO2
Experiment 1.4	To illustrate a Music Streaming Service LLD using UML modeling, OOP concepts in Java, SOLID principles, and suitable design patterns (Singleton, Factory Method, Observer, Strategy) to support playing songs, playlist creation, artist following, and personalized recommendations while considering basic performance factors.		CO1, CO2
UNIT 2	High-Level System Design (HLD)	10 Hours	CO Mapping

Chapter 2.1 —Networks Fundamentals

Types of Servers (Web, Application, Database, Cache),HTTP vs HTTPS, REST Communication Formats (JSON), Proxy Server vs Reverse Proxy Server, TCP vs UDP (High-Level Understanding), DNS,URL-to-Website Flow (What happens when you type a URL), CORS (Cross-Origin Resource Sharing).

Chapter 2.2 — Scalability & System Components

Vertical Scaling , Horizontal Scaling, Load Balancers (L4 vs L7), Stateless vs Stateful Architecture, Rate Limiting (Token Bucket, Leaky Bucket), Fault Tolerance & High Availability , Auto Scaling (High-Level Overview), Redundancy (Basics of avoiding single points of failure), CAP Theorem (High-Level), Consistency Models (Strong vs Eventual).

Chapter 2.3 — API Design & Communication

Overview of REST Architecture, API Endpoint Design (Resources, URIs, Naming Conventions), HTTP Methods, HTTP Status Codes, Request & Response Structure, API Versioning Strategies, Authentication & Authorization (High-Level Overview), Idempotency in API Design (High-Level), Rate Limiting (API-level), Microservices Communication Basics (Sync vs Async), Client–Server Interaction Model (High-Level), Overview of gRPC vs REST (Simple comparison).

Chapter 2.4 — Caching & Content Delivery

Redis basics, Cache-aside Pattern, Write-through vs Write-back (High-Level) , Cache invalidation strategies, Cache Eviction Policies, Distributed Cache (Redis Cluster), CDN concepts & use cases.

Experiment 2.1	To execute a URL-to-Website Request Flow Module that simulates DNS lookup, HTTP/HTTPS request handling, and reverse proxy behavior, while modeling the request response structure using UML and implementing a basic networking simulation in Programming Language.		CO1, CO2, CO3
Experiment 2.2	To analyze a Load-Balancing Simulation demonstrating horizontal scaling using stateless servers, observing changes in throughput and fault tolerance under increased user load using Java multi-threading.		CO3
Experiment 2.3	To implement REST APIs for a Playlist & Recommendation Service using HTTP methods, response codes, API-level rate limiting, and Redis cache-aside strategy, while applying SOLID and Strategy/Observer patterns to analyze scalability improvement.		CO2, CO3, CO4
Unit 3:	Advanced Distributed Systems & Observability (HLD)	10 Hours	CO Mapping

Chapter 3.1 — Database Scalability & Hashing

SQL vs NoSQL (High-Level Comparison), Normalized vs Denormalized Data, Database Replication (Master–Slave, Master–Master), Database Sharding (Range, Hash, Directory-based), Hashing Concepts (Consistent Hashing), Indexing Basics, Partitioning Strategies, Distributed Transactions (High-Level: 2PC, Saga),Read/Write Splitting, Connection Pooling Basics.

Chapter 3.2 — Message Queues & Event-Driven Systems

Message Queues (Kafka, RabbitMQ — Concepts Only), Event-Driven Architectures (Producers, Consumers, Topics), Pub/Sub Model, Event Sourcing (High-Level), Asynchronous Communication, Distributed Systems Basics, Leader Election (High-Level), Heartbeats & Health Checks, Circuit Breaker Pattern Retry Strategies & Dead Letter Queues.

Chapter 3.3 — General Architecture Patterns

Monolith vs Microservices, Modular / Layered Architecture, Hexagonal Architecture (High-Level), Clean Architecture (Intro), API Gateway Concepts, Routing, Aggregation, Authentication via Gateway, BFF (Backend For Frontend) Pattern, Service Registry & Discovery (Eureka/Consul — Conceptual), Sidecar Pattern (Service Mesh Basics: Istio/Envoy), Observability Basics (Logs, Metrics, Tracing).

Experiment 3.1	To analyze the performance difference between a single relational database and a sharded + replicated database setup by implementing hash-based sharding for users/orders, enabling read/write splitting using Java, and examining query latency, consistency behavior, and failure handling using UML diagrams and database logs.	CO3, CO4
Experiment 3.2	To evaluate an asynchronous Order Processing workflow by implementing a Producer–Consumer model using a message queue where producers publish order events, consumers update inventory and send notifications, and the system uses retry with dead-letter logic to validate event consistency compared to synchronous REST-based processing.	CO3, CO4
Experiment 3.3	Lab Based Mini Project	CO2, CO3, CO4, CO5



Netiquette Guidelines

Mark as done



Computer Requirements

Mark as done

Hardware Requirements

Component	Minimum Requirement
Processor	Dual-Core CPU (Intel i3 or equivalent)
RAM	8 GB
Storage	256 GB HDD / SSD
Display	14" screen (1366×768 resolution)
Internet	Basic broadband connection
Accessories	Keyboard, Mouse, Headphones (optional)

Software Requirements

Category	Tools / Software
Operating System	Windows 10 or above / Linux (Ubuntu) / macOS
Programming Tools	VS Code / IntelliJ / PyCharm / Eclipse
Browsers	Google Chrome / Mozilla Firefox
Database Tools	MySQL / MongoDB (any one)
API Development	Postman
Modeling Tools	Draw.io / StarUML (any one)



Announcements

Jump to...

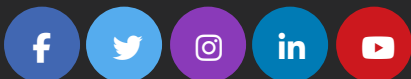


Unit-1 (Theory) ►

Contact us



Follow us



You are logged in as MOHD SHAN . (Log out)